

Praktikum - Betriebssysteme - seL4 & TRENTOS

Abschlussvortrag

Entwicklung einer Komponente zur
verschlüsselten Kommunikation

München, 7. Februar 2023

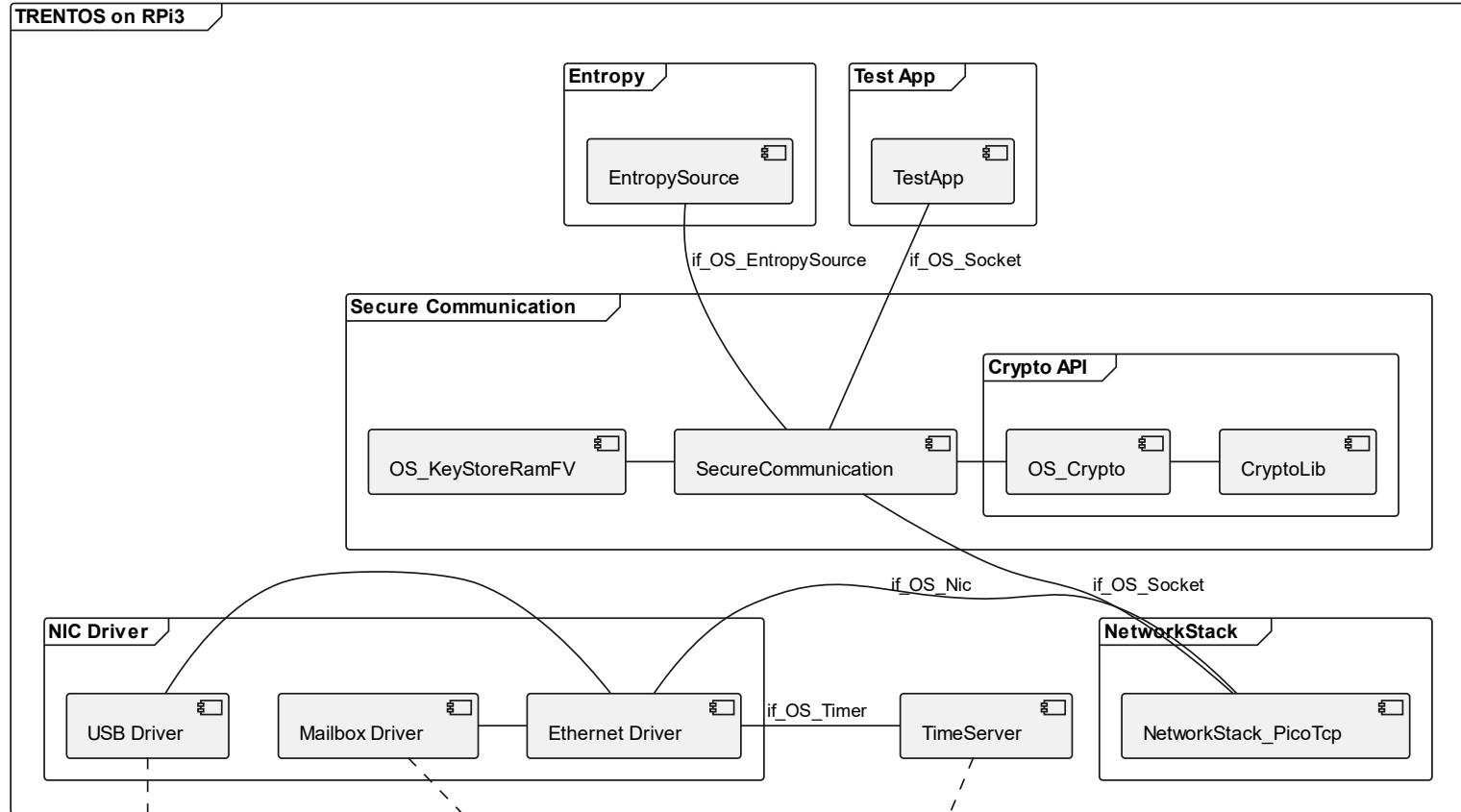
Jakob Kukla, Moritz Beckel

Ziele

1. Entwicklung einer rein **Software-basierenden** Lösung (Moritz Beckel)
2. Entwicklung einer **TPM-basierenden** Lösung (Jakob Kukla)
3. **Performanzvergleich** beider Lösungen

Software-basierende Lösung

1. Entwicklung einer Proxy Komponente, welche mithilfe von `if_OS_Socket`, eingehende Anfragen, an das Netzwerkstack weiterleitet
2. Verschlüsselung der weitergeleiteten Daten und Generierung von Schlüsseln mithilfe der `Crypto API`
3. Speichern der Schlüssel mit der `Keystore API`
4. Austausch von Schlüsseln über das `PEM Format` mithilfe von `mbedtls`



If_OS_Socket

Ziel: Weiterleitung der Socket-Aufrufe an die eigentliche Implementierung

Problem: Unterschiedliche Methodensignatur

```
OS_Error_t  
secureCommunication_rpc_socket_write(const int handle, size_t *const pLen)
```



```
OS_Error_t  
OS_Socket_write(  
    const OS_Socket_Handle_t handle,      const void* const buf,  
    const size_t requestedLen, size_t* const actualLen);
```

If_OS_Socket

if_OS_Socket besitzt **keinen expliziten Dataport**, Verbindung über **seL4RPCDataport**

```
connection seL4RPCDataport conn_##inst_nw_stack##_##inst_user##_rpc(  
  from inst_user.inst_user_field_prefix##_rpc,  
  to   inst_nw_stack.inst_nw_stack_field_prefix##_rpc);
```

Zugriff auf Buffer möglich über:

```
<from_interface>_buf(<interface>_get_sender_id());
```

Verschlüsselung

- Schlüssel werden Out of band übertragen
- Asymmetrische Verschlüsselung
- **RSA PKCS#1 OAEP** als Verschlüsselungsalgorithmus
- **RSA-2048** Schlüssel verwendet
- **SHA256** als Hashalgorithmus

256 Byte großen Blöcke verschlüsselt, davon sind maximal 190 Byte Daten

Verschlüsselung – Senden

Einschränkung: **Buffergröße** auf **190 Byte** beschränkt, aber **behält** trotzdem **Funktionalität** von Socket Interface **bei**: geschriebene Länge wird dem Sender mitgeteilt

```
OS_Error_t
secureCommunication_rpc_socket_write(int handle, size_t *pLen)
{
    size_t expectedLen = min(*pLen, 190);
    encrypt();
    OS_Socket_write();
    ...
}
```


Verschlüsselung – Empfangen

Beim Lesen von Nachrichten wird **sichergestellt**, dass **ein ganzer Block empfangen**, wird, sodass **nichts gepuffert** werden muss und **direkt entschlüsselt** werden kann.

```
OS_Error_t
secureCommunication_rpc_socket_read(int handle, size_t *pLen)
{
    OS_Socket_read();
    decrypt();
    assert(receivedLength == 256);
    ...
}
```

Schlüsselformat

RSA Schlüssel werden meistens im **PEM** Format dargestellt:

```
-----BEGIN PUBLIC KEY-----
```

```
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAhT+IEvoBIItgpNjA5ZoCs  
1VlPLqLMdTxNc1C5/nkMSuZHWQrxxaDSWo45psWRHwYd+nDH7ZvqHXM0+0yOprnL  
evdgcpxJgW8tnr7PzFPKCIJoJsgpBqEXWZkuU/6zBSMGRz1P9L3QKgx+KTvAj4RF  
YswiCsBwKRnoKqBYwK5gy1V4HxtZAD8iY2ir/xwnIcpgY6aDTWS7q8Rjpia3kiu1  
LvYvIVPaDiNr1rEOfemkEGIyCUriXU2DgTxFDDXn9mmbOCV8AttepHC4Mf7b1n1U  
sk4Rn8dGo/9eqV0gSN/yDT/GyWKCs79zwQ0dsjddrm3/fyCfVAAip4/7YaRcynjb  
XwIDAQAB
```

```
-----END PUBLIC KEY-----
```

Schlüsselformat

Konvertierung der Schlüsselformate mithilfe der [mbedtls](#) Bibliothek

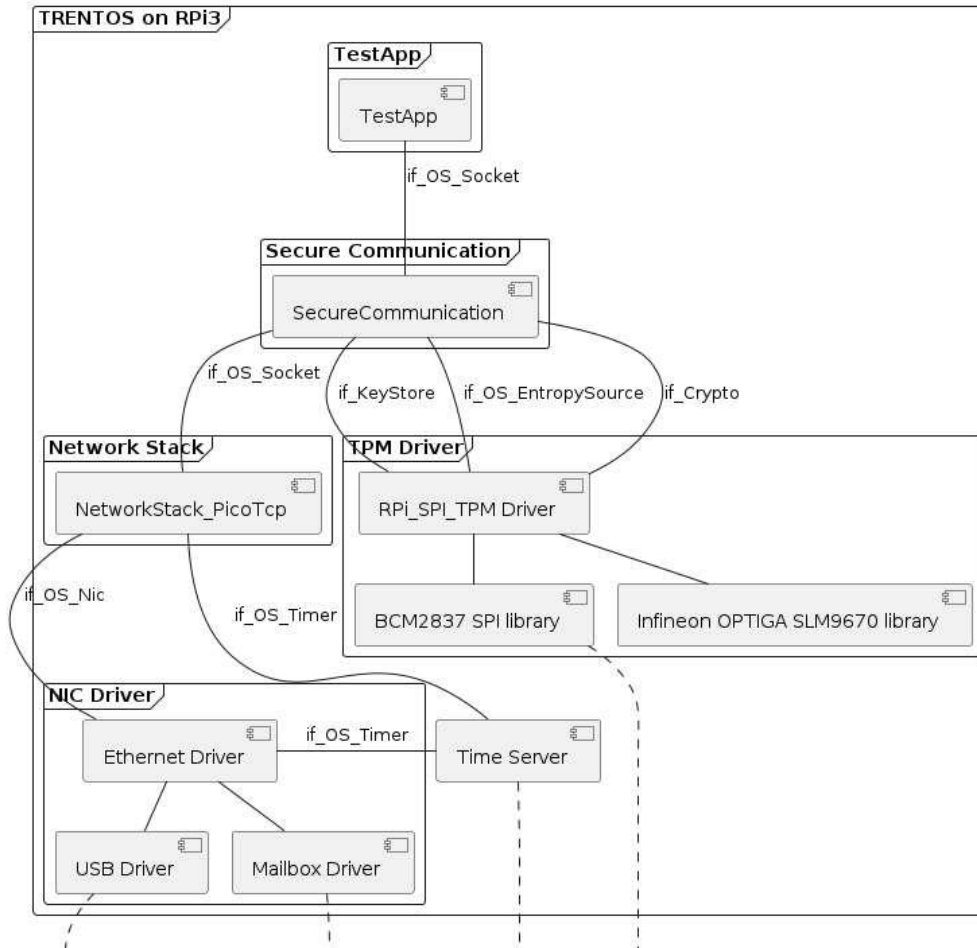
```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAhT+IEvoBItgpnjA5ZoCs
1VlPLqLMDTxNc1C5/nkMSuZHWQrxxaDSWo45psWRHwYd+nDH7ZvqHXM0+OyOprnL
evdgcpxjgW8tnr7PzFPKCIJoJsgpBqEXWZkuU/6zBSMGRz1P9L3QKgx+KTvAj4RF
YswiCsBwKRnoKqBYwK5gy1V4HxtZAD8iY2ir/xwnIcpgY6aDTWS7q8Rjpa3kiu1
LvYvIVPaDiNr1rEOfemkEGiYCUriXU2DgTxFDDXn9mmboOCV8AttepHC4Mf7b1nLU
sk4Rn8dGo/9eqV0gSN/yDT/GyWKcs79zwQ0dsjddrm3/fyCfVAAip4/7YaRcynjb
XwIDAQAB
-----END PUBLIC KEY-----
```

```
/**
 * Struct for RSA public key data.
 */
typedef struct
{
    uint8_t nBytes[OS_CryptoKey_SIZE_RSA_MAX];
    uint32_t nLen;
    uint8_t eBytes[OS_CryptoKey_SIZE_RSA_MAX];
    uint32_t eLen;
} OS_CryptoKey_RsaRub_t;
```

Nachteile der Software-basierten Lösung

- Crypto API wird in Library Mode verwendet für die RSA Verschlüsselung und zur Schlüsselgenerierung, **wenig Sicherheit** (da alle Schlüssel lokal)
- EntropySource Komponente verwendet **keine echte Entropie**
- Zum Speichern der Schlüssel wird KeystoreRamFV Komponente verwendet, Speicher ist **nicht persistent**

TPM-basierte Lösung



Überblick - Workflow

1. (Beim **ersten** Aufruf eines leeren TPMs wird **SRK** generiert)
2. RSA Key wird aus **Keystore** geladen
3. RSA Key wird aus **system_config.h** importiert und im **Keystore** gespeichert
4. RSA Key wird **generiert** und im **Keystore** gespeichert
5. **Ver- / Entschlüsseln** auf Nachfrage von SecureCommunication component

Storage Root Key (SRK)

- Eigentlich nur **Storage Key**
- Liegt in der **TPM Owner Hierarchie**
- Verwendet zur **Generierung** und zum **Speichern / Laden** von Schlüssel
 - **Schnellere** Key Generierung
 - **Sichere** Speicherung

Kommunikation mit TPM

- SPI und GPIO
 - abstrahiert durch [bcm2837](#)
- TPM2-TIS (TPM Interface Specification)
 - abstrahiert durch [wolfTPM](#)
- Exportieren und Importieren von PEM
 - [wolfSSL](#) in Kombination mit [mbedtls](#)

Herausforderungen beim Kommunizieren über SPI

- RPi **Pin 18** muss **high** gepulst werden
 - *enable_jtag_gpio=0* in *config.txt*
- Keine Möglichkeit herauszufinden, ob Transfer **abgeschlossen**
- **SLB9670** braucht 10 μ s zwischen einzelnen Transfers um Fehler zu vermeiden
 - TimeServer benötigt, aber ungenau

wolfTPM Portierung

- wolfTPM bietet IO-Callback an, um einfach von HW zu abstrahieren

```
/* IO Callback */
int TPM2_IoCb(TPM2_CTX *ctx, const byte *txBuf, byte *rxBuf, word16 xferSz,
              void *userCtx) {
    bcm2837_spi_transferrnb((char *)txBuf, (char *)rxBuf, xferSz);
    util_sleep(20);

    (void)userCtx;
    (void)ctx;

    // FIXME: There is no way of knowing if the transfer was successful
    return TPM_RC_SUCCESS;
}
```

Interfaces

- Funktionsaufrufe über [RPC](#)
- Große Daten mittels [Dataport](#) übertragen
- Bereitstellung von [Wrapper Libraries](#), zur einfachen Konsumierung
 - [TPM_Crypto](#)
 - [TPM_Keystore](#)

Interfaces – Beispiel if_Keystore

- Interface **Definition**
- Einziger Parameter über **RPC** ist das Handle zur Adressierung

```
procedure if_Keystore {  
    include "OS_Error.h";  
  
    OS_Error_t loadKey(  
        in int id  
    );  
  
    OS_Error_t storeKey(  
        in int id  
    );  
};
```

Interfaces – Beispiel if_Keystore

- Wrapper Methode in `TPM_Keystore` library:

```
OS_Error_t TPM_Keystore_storeKey(const TPM_Keystore_Handle_t *ctx,
    size_t handle, TPM_Crypto_Key_t *key) {
    int rc;

    memcpy(OS_Dataport_getBuf(ctx->dataport), key, TPM_CRYPT0_KEY_SIZE);

    rc = ctx->storeKey(TPM_RSA_BASE_HANDLE + handle);

    memcpy(key, OS_Dataport_getBuf(ctx->dataport), TPM_CRYPT0_KEY_SIZE);

    return rc;
}
```

if_Keystore

- Keys werden im **NV-Ram** des TPMs **persistent** gespeichert
 - Parent Key ist **Storage Root Key (SRK)**
 - Ebenfalls im **NV-Ram** gespeichert
- Adressierung erfolgt mittels **Handle**
- **Private Key** verlässt niemals das TPM

Interfaces – Beispiel if_Crypto

- Aufbau ähnlich wie bei `if_Keystore`
- Performance Optimierung durch Entfernen redundanter `memcpy()` Aufrufe

```
procedure if_Crypto {  
    include "OS_Error.h";  
  
    OS_Error_t generateKey();  
  
    OS_Error_t importPublic();  
  
    OS_Error_t importPrivate();  
  
    OS_Error_t exportPublicPem(out size_t size);  
  
    OS_Error_t encrypt(in size_t input_size, out size_t output_size);  
  
    OS_Error_t decrypt(in size_t input_size, out size_t output_size);  
};
```


Interfaces – Beispiel if_Crypto

- Beispiel
[crypto_rpc_encrypt\(\)](#)
 Methode

```
OS_Error_t crypto_rpc_encrypt(size_t input_size, size_t *output_size) {
    int tpm_rc;
    unsigned char output[OS_Dataport_getSize(c_port)];
    WOLFTPM2_KEY *key = OS_Dataport_getBuf(c_port);

    tpm_rc =
        wolfTPM2_RsaEncrypt(&dev, key, TPM_ALG_OAEP,
                           OS_Dataport_getBuf(c_port) + sizeof(WOLFTPM2_HANDLE),
                           (int)input_size, output, (int *)output_size);
    if (tpm_rc != TPM_RC_SUCCESS) { ...
    }

    if (OS_Dataport_getSize(c_port) < *output_size) { ...
    }

    memcpy(OS_Dataport_getBuf(c_port), output, *output_size);

    return OS_SUCCESS;
}
```

Interfaces – Beispiel if_Crypto

- Beispiel
`crypto_rpc_generateKey()`
Methode

```
OS_Error_t crypto_rpc_generateKey() {  
    int rc, tpm_rc;  
    WOLFTPM2_KEY *key = OS_Dataport_getBuf(c_port);  
    WOLFTPM2_KEY srk;  
    TPMT_PUBLIC template;  
  
    rc = tpm_int_get_or_create_srk(&srk);  
    if (rc != OS_SUCCESS) { ...  
    }  
  
    tpm_rc = wolfTPM2_GetKeyTemplate_RSA(  
        &template, TPMA_OBJECT_sensitiveDataOrigin | TPMA_OBJECT_userWithAuth |  
        TPMA_OBJECT_decrypt | TPMA_OBJECT_noDA);  
    if (tpm_rc != TPM_RC_SUCCESS) { ...  
    }  
  
    tpm_rc = wolfTPM2_CreateAndLoadKey(&dev, key, &srk.handle, &template, NULL, 0);  
    if (tpm_rc != TPM_RC_SUCCESS) { ...  
    }  
  
    return OS_SUCCESS;  
}
```

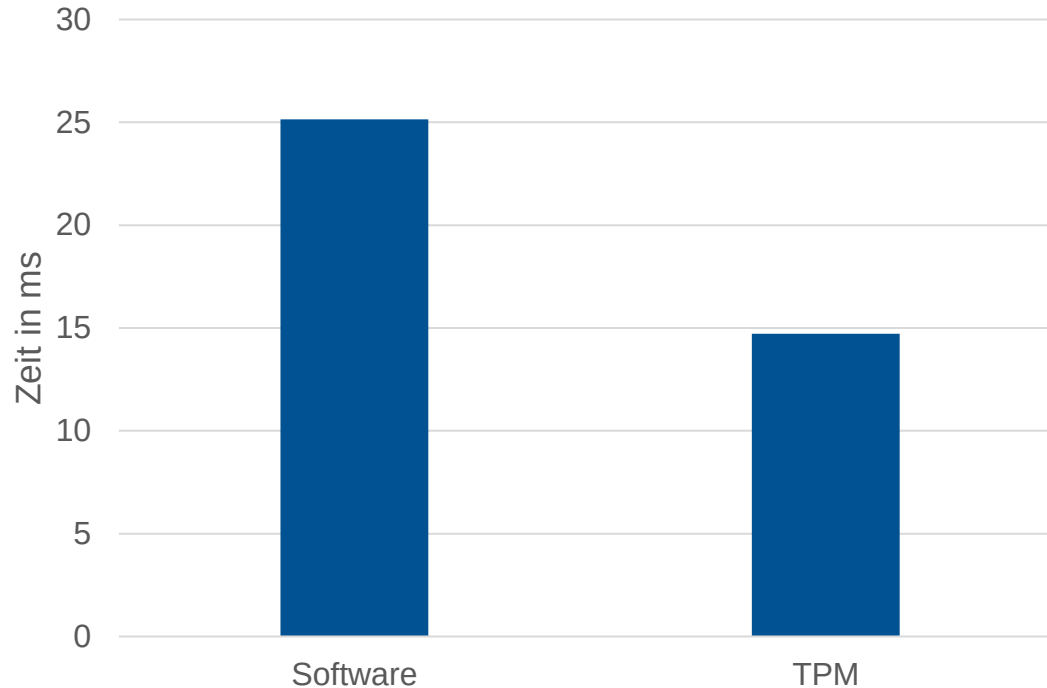
Verbesserungsmöglichkeiten

- Parameter Encryption
 - Verschlüsseln der TPM Kommando Parameter
 - SPI **man-in-the-middle** attack nicht mehr möglich
- RSA Client / Server Schlüssel haben **keine Authentication**
 - Damit externe Schlüssel auch gespeichert und geladen werden können

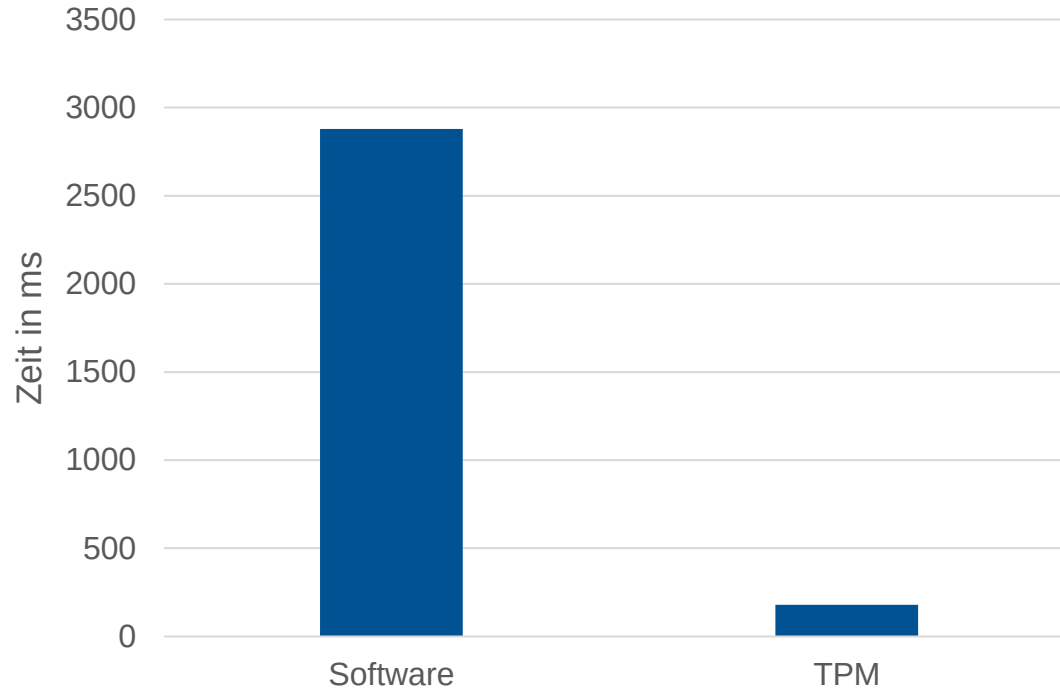
Benchmark

- Messung der **Ver- und Entschlüsselung** mit und ohne TPM
- **Verschlüsselung** wird **100 Mal** in Folge ausgeführt
- **Entschlüsselung** nur **10 Mal**
- **Schlüsselgenerierung** nur ein Mal
- Insgesamt werden alle Operationen 5 mal ausgeführt für eine geringere Abweichung

Benchmark – Verschlüsselung



Benchmark – Entschlüsselung



Benchmark – Schlüsselgenerierung

